

**DESIGN AND IMPLEMENTATION
OF
2-CHANNEL DMA CONTROLLER
FOR
RISC-V SoC INTEGRATION**

FINAL REPORT

Submitted by

Manvith Shivaram Bangera

Department of

Electronics and Communication Engineering

Under the Guidance of

N J Hariprasad

Academic Year 2025–2026

April 2026

1 Background Study

Direct Memory Access (DMA) is a hardware mechanism that enables data transfer between memory locations without continuous processor intervention.

In traditional processor-based systems, the CPU performs each memory read and write operation, increasing execution overhead and reducing system efficiency.

DMA improves performance by independently managing memory transfers while the processor performs other tasks.

This project implements a dual-channel DMA controller integrated into a RISC-V SoC.

The DMA supports:

- Single transfer mode
- Burst transfer mode
- Dual-channel architecture
- Arbitration-based shared bus access

2 Module Specification

2.1 Port Description

Signal	Direction	Width	Description
clk	Input	1	System clock
rst	Input	1	System reset
cfg_awvalid	Input	1	AXI write address valid
cfg_wvalid	Input	1	AXI write data valid
cfg_arvalid	Input	1	AXI read address valid
cfg_rdata	Output	32	AXI read data
count_o	Output	4	DMA status compatibility output

Table 1: DMA Peripheral Port Description

3 DMA Register Map

The DMA controller is memory-mapped into the SoC peripheral address space.

Register	Offset
DMA_SRC	0x00
DMA_DST	0x04
DMA_LEN	0x08
DMA_MODE	0x0C
DMA_CTRL	0x10
DMA_STATUS	0x14

Table 2: DMA Register Mapping

Base Address:

0x95000000

4 Working Modes

4.1 Single Transfer Mode

In single mode:

- One word is read from source memory
- One word is written to destination memory
- Source address increments by 4 bytes
- Destination address increments by 4 bytes
- Process repeats until transfer completion

Transfer sequence:

READ -> WRITE -> READ -> WRITE

4.2 Burst Transfer Mode

In burst mode:

- Multiple reads are performed first
- Data is stored in burst buffer
- Buffered data is written to destination
- Reduces bus switching overhead

Transfer sequence:

READ READ READ READ -> WRITE WRITE WRITE WRITE

5 RTL Design

5.1 Architecture Overview

The DMA architecture consists of:

- DMA Channel 0
- DMA Channel 1
- Round-robin Arbiter
- Shared Bus Interface
- Internal Burst Buffer

Architecture:

```
Channel 0 ----\
                --> Arbiter --> Shared Bus --> Memory
Channel 1 ----/
```

6 RTL Schematic

The elaborated RTL schematic generated in Vivado is shown below.

The schematic verifies:

- Dual DMA channel integration
- Shared bus architecture
- Arbiter-based bus control
- Multiplexer-based bus routing
- Synthesizable hardware structure

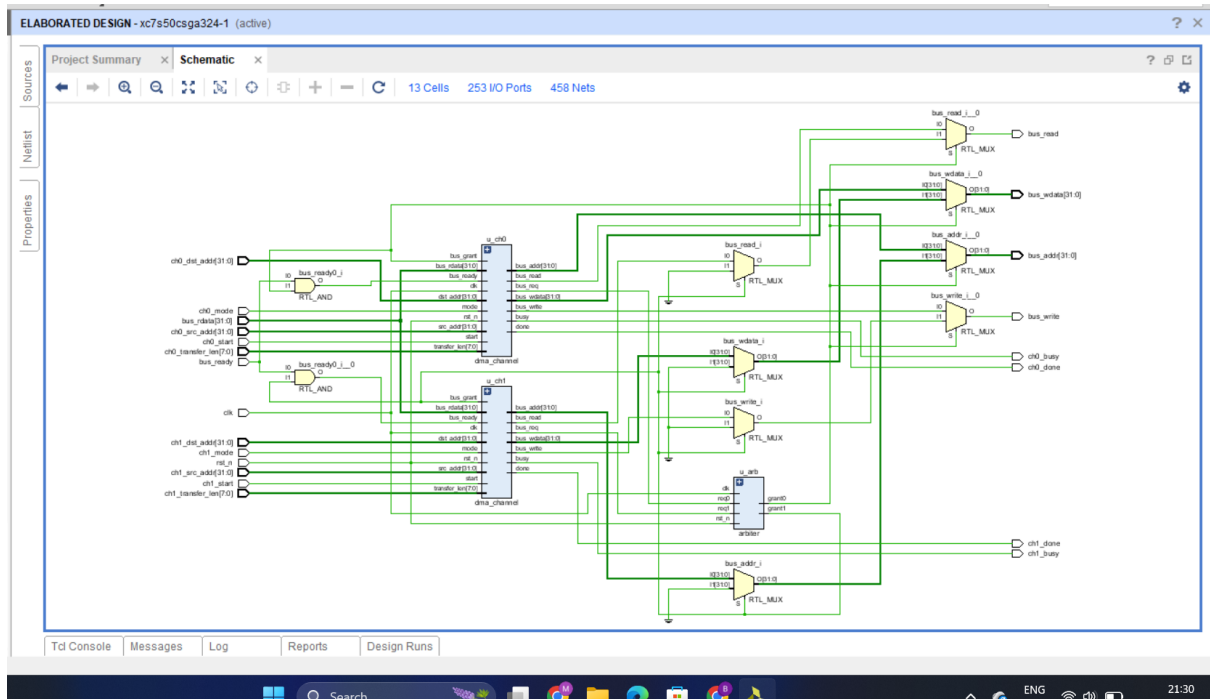


Figure 1: Vivado Elaborated RTL Schematic of DMA Architecture

7 System Block Diagram

The integrated DMA-based RISC-V SoC architecture is shown below.

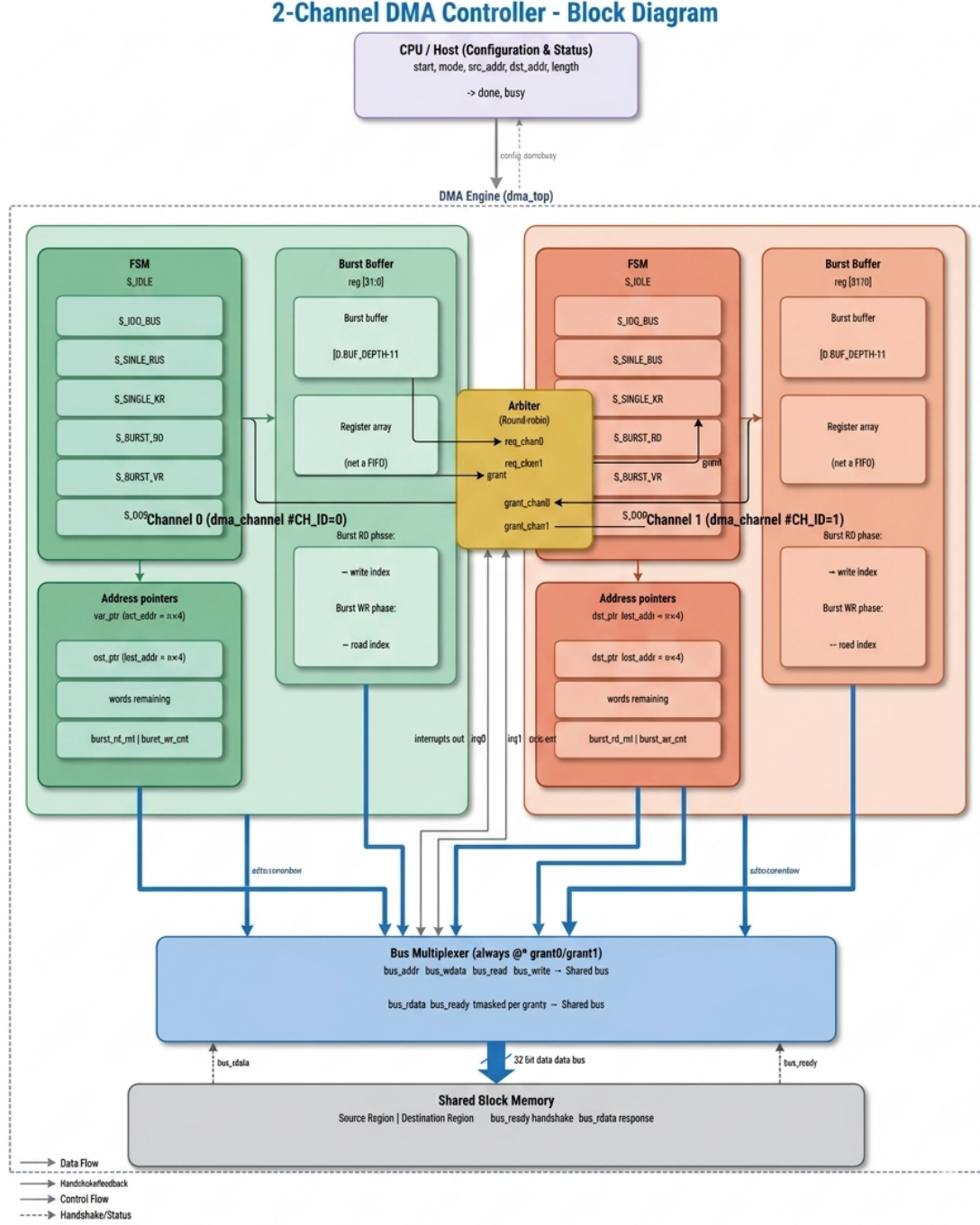


Figure 2: Integrated DMA Controller in RISC-V SoC

8 RISC-V SoC Integration

The DMA controller was integrated into the RISC-V SoC by modifying only:

- counter.v
- counter_defs.v

The existing SoC architecture remained unchanged.

Preserved modules:

- UART interface
- Firmware loader
- Memory interconnect
- Top-level SoC module

The existing counter peripheral wrapper was reused and internally modified to implement DMA functionality while preserving the original AXI peripheral slot.

9 Implementation

Modules used:

- counter.v
- counter_defs.v
- dma_top.v
- dma_channel.v
- arbiter.v

Tools used:

- Vivado Design Suite
- WSL Ubuntu
- RISC-V GCC Toolchain
- FPGA Development Board

10 Verification

Verification was performed using:

- RTL simulation
- FPGA synthesis
- Bitstream generation
- Hardware execution in WSL terminal

11 Test Cases

Test Case	Mode	Result
TC1	Single Transfer Mode	Pass
TC2	Burst Transfer Mode	Pass
TC3	Dual Channel Arbitration	Pass
TC4	Hardware Output Verification	Pass

Table 3: DMA Verification Test Cases

12 RTL Waveform Analysis

The simulation waveform verifies correct DMA transfer operation.

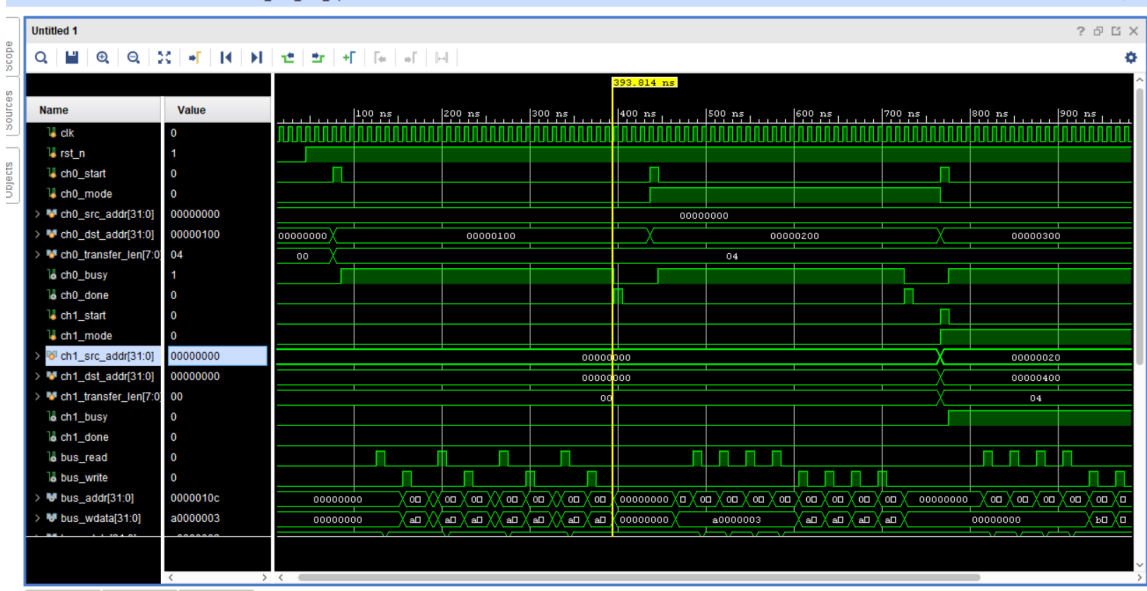


Figure 3: DMA Transfer Simulation Waveform

Observations:

- Address increments correctly
- Bus read operations verified
- Bus write operations verified
- Arbitration functioning correctly

13 Hardware Output

The following figures show actual DMA execution output in WSL.

13.1 Single Transfer Mode Output

```
/home/manvith/riscv_soc/fpga/boolean/run/console-uart.py -t uart -d /dev/ttyUSB1 -b 1000000 -f dma_test.elf

=====
Dual-Channel DMA Controller Demo
RISC-V SoC | Boolean Board
=====
DMA Start

-- SINGLE Mode --
One word read then written at a time

Source (random this run):
[0x95000020] = 55
[0x95000024] = 133
[0x95000028] = 125
[0x9500002C] = 246
Destination BEFORE DMA (data from previous run):
[0x95000060] = 0
[0x95000064] = 0
[0x95000068] = 0
[0x9500006C] = 0
Starting SINGLE DMA Transfer...
SRC=0x95000020 DST=0x95000060 LEN=4 words
Destination AFTER DMA (overwritten with new data):
[0x95000060] = 55
[0x95000064] = 133
[0x95000068] = 125
[0x9500006C] = 246
SINGLE Mode: PASS
```

Figure 4: Single Mode DMA Output

Observation:

Destination memory is successfully overwritten with source data.

13.2 Burst Transfer Mode Output

```
manvith@DESKTOP-ITEHSJA: × + v

-- BURST Mode --
All reads into buffer first, then all writes at once

Source (random this run):
[0x95000020] = 43
[0x95000024] = 51
[0x95000028] = 63
[0x9500002C] = 38
[0x95000030] = 230
[0x95000034] = 197
[0x95000038] = 199
[0x9500003C] = 110
Destination BEFORE DMA (data from previous run):
[0x95000060] = 55
[0x95000064] = 133
[0x95000068] = 125
[0x9500006C] = 246
[0x95000070] = 0
[0x95000074] = 0
[0x95000078] = 0
[0x9500007C] = 0
Starting BURST DMA Transfer...
SRC=0x95000020 DST=0x95000060 LEN=8 words
Destination AFTER DMA (overwritten with new data):
[0x95000060] = 43
[0x95000064] = 51
[0x95000068] = 63
[0x9500006C] = 38
[0x95000070] = 230
[0x95000074] = 197
[0x95000078] = 199
[0x9500007C] = 110
BURST Mode: PASS

=====
DMA Copy Successful
DMA Done
=====
```

Figure 5: Burst Mode DMA Output

Observation:

Burst transfer correctly copies all source data into destination memory.

14 Results and Analysis

The hardware execution confirms:

- Random source data generation
- Previous destination data persistence
- Correct overwrite behavior
- Successful single transfer mode
- Successful burst transfer mode

The destination memory after DMA exactly matches the source memory.
This verifies:

- Correct source addressing
- Correct destination addressing
- Correct transfer length
- Correct DMA completion signaling

15 Challenges and Debugging

Challenges faced:

- Counter wrapper repurposing for DMA functionality
- Register mapping integration
- UART communication verification
- USB serial interface configuration in WSL
- Bitstream debugging
- Hardware execution validation

16 Conclusion

A complete dual-channel DMA controller supporting both single and burst transfer modes was successfully designed and integrated into a RISC-V SoC.

The existing counter peripheral wrapper was reused and internally modified to implement DMA functionality while preserving the original AXI4-Lite interface and SoC architecture.

The integrated hardware was synthesized, programmed on FPGA, and verified through actual hardware execution.

The project successfully demonstrated:

- Single transfer mode
- Burst transfer mode
- Dual-channel arbitration
- Hardware-level execution
- Correct memory-to-memory transfer

The implementation validates the functional correctness of the DMA controller in real hardware.